RV College of Engineering®
Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

Blockchain Voting with Agentic Accessibility

An Interdisciplinary Project Report (XX367P)

Submitted by,

Srikar Rao H M	**1RV23CS245**
Srinidhi H	**1RV23CS246**
S Vignajit	**1RV22EI043**
Jayathi R	**1RV23AS017**

Under the guidance of

Dr. N S Narahari
Visiting Professor
Dept. of IEM
R V College of Engineering®

.

In partial fulfillment of the requirements for the degree of Bachelor of Engineering in respective departments

2026

RV College of Engineering®, Bengaluru

(*Autonomous institution affiliated to VTU, Belagavi*)

CERTIFICATE

Certified that the interdisciplinary project (XX367P) work titled ***Blockchain Voting with Agentic Accessibility*** is carried out by of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering** in respective departments during the year 2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the interdisciplinary project report deposited in the departmental library. The interdisciplinary project report has been approved as it satisfies the academic requirements in respect of interdisciplinary project work prescribed by the institution for the said degree.

Dr. N S Narahari Guide	Rajeswara Rao K V S Head of the Department
Dr. M.V. Renukadevi Dean Academics	Dr. K. N. Subramanya Principal

External Viva

Name of Examiners	Signature with Date
1.	
2.	

DECLARATION

We, **Srikar Rao H M**, **Srinidhi H**, **S Vignajit**, **Jayathi R** students of sixth semester B.E., RV College of Engineering, Bengaluru, hereby declare that the interdisciplinary project titled '**Blockchain Voting with Agentic Accessibility**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in respective departments during the year 2026.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and We will be one of the authors of the same.

Place: Bengaluru

Date:

Name	Signature
1. Srikar Rao H M(1RV23CS245)	
2. Srinidhi H(1RV23CS246)	
3. S Vignajit(1RV22EI043)	
4. Jayathi R(1RV23AS017)	

ACKNOWLEDGEMENTS

We are indebted to our guide, **Dr. N S Narahari**, Visiting Professor, Department of IEM, RVCE for the wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

We also express our gratitude to our panel member **S. G. Raghavendra Prasad**, Assistant Professor, Department of ISE, RVCE for the valuable comments and suggestions during the phase evaluations.

Our gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

Our sincere thanks to **Rajeswara Rao K V S**, Professor and Head, Department of Industrial Engineering & Management, RVCE for the support and encouragement.

We are deeply grateful to **Dr. M.V. Renukadevi**, Professor and Dean Academics, RVCE, for the continuous support in the successful execution of this Interdisciplinary project.

We express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

ABSTRACT

This work addresses the need for privacy-preserving, verifiable electronic voting in contexts where centralized infrastructure is unreliable or undesirable. Conventional e-voting systems often expose identity–choice linkage through timing correlations, single-vote on-chain writes, or centralized identity handling. These limitations create privacy risks and reduce trust. This report addresses those issues by separating identity tracking from vote recording, using off-chain batching and shuffling with on-chain tallying, and by adopting a local blockchain model suited for edge deployments.

The objectives are to design and implement a voting workflow that preserves anonymity, prevents double voting, and remains auditable. The methodology uses hash-based identity verification, a two-transaction batching model, and a state-driven smart contract that enforces election phases. The system integrates an agentic, voice-guided voting flow to improve accessibility while maintaining strict confirmation logic and neutral guidance.

Simulation and validation are performed on a local blockchain environment to reflect the intended deployment path for modular, edge-device voting terminals. The software stack includes a web-based voting interface, a relayer service for batch submission, and a smart contract that maintains tallies and election state. Test cases include repeated voting attempts, invalid candidates, state transitions, and batch submission thresholds. Results show that identity-to-choice linkage is removed on-chain, double voting is prevented, and tallies remain verifiable through contract state.

Hardware emulation is not performed in this phase; the prototype focuses on software and local-chain validation. Future work includes cryptographic proofs for shuffle correctness, hardened key management, and formal audits to further reduce trust in the relayer and improve deployment readiness.

CONTENTS

Abstract i

List of Figures v

List of Tables vi

Abbreviations vii

1 Introduction 1

1.1 Introduction . . . 2

1.2 Literature Review . . . 2

1.2.1 Review of Related Works . . . 2

1.3 Motivation . . . 3

1.4 Problem Statement . . . 3

1.5 Objectives . . . 3

1.6 Brief Methodology of the project . . . 3

1.7 Assumptions made / Constraints of the project . . . 4

1.8 Organization of the Report . . . 4

2 Design of Blockchain-Based Voting Architecture 6

2.1 System Overview . . . 7

2.1.1 Voter Interface . . . 7

2.1.2 Off-Chain Buffer and Shuffle Module . . . 7

2.1.3 Relayer Service . . . 8

2.1.4 Smart Contract . . . 8

2.2 System Flow Description . . . 8

2.3 Data Handling and Privacy . . . 10

2.3.1 Threat Model Considerations . . . 10

2.3.2 Local Blockchain Rationale . . . 10

2.4 Tools and Technologies Used . . . 11

2.4.1 Smart Contract Platform . . . 11

2.4.2 Off-Chain Components . . . 11

2.4.3 User Interface Framework 11
2.5 Use of Acronyms and Glossaries 12

3 System Methodology and Implementation 13
3.1 Specifications for the Design 14
3.2 Technology Stack and Pre-Analysis 15
3.2.1 Threat Model 15
3.2.2 Technology Stack 15
3.3 Design Methodology 15
3.3.1 Smart Contract Logic 15
3.3.2 Off-Chain Processing and Design Equations 15
3.4 Local Chain Rationale 16

4 Prototype Implementation and Validation 17
4.1 Implemented Modules 18
4.1.1 Voter Interface and Agentic Flow 18
4.1.2 Off-Chain Batching and Shuffle Module 18
4.1.3 Smart Contract 18
4.2 Security and Anonymity Coverage 19
4.3 Validation Status 19

5 Future Work 21
5.1 Validation Results 22
5.1.1 Test Cases and Outcomes 22
5.1.2 Anonymity Verification 22
5.2 Interpretation of Results 23
5.3 Future Work 23
5.3.1 Privacy Hardening 23
5.3.2 Deployment Readiness 23
5.3.3 Hardware Integration 24

A Code 25
A.1 Smart Contract and Batch Processing 26
A.1.1 AnonymousBallot Contract 26

A.1.2 Batching Logic 28
A.1.3 Secure Shuffle 30
A.1.4 Vote Submission Flow 30

LIST OF FIGURES

1.1 Methodology Flow 4

2.1 Authentication and Identity Hashing 8
2.2 Candidate Selection 9
2.3 Standard Visual Interface 11
2.4 Voice-Guided Interface 12

3.1 Blockchain live dashboard 14
3.2 Off-chain batching and shuffle workflow prior to on-chain submission . . . 16

4.1 Security and anonymity coverage across system layers 19

5.1 Microcontroller ESP32 (a) 24
5.2 Microcontroller ESP32 (b) 24

LIST OF TABLES

5.1 Validation test cases and outcomes 22

ABBREVIATIONS

API Application Programming Interface

EVM Electronic Voting Machine

HTTPS HyperText Transfer Protocol Secure

ID Identity

REST REpresentational State Transfer

RFID Radio Frequency IDentifier

RPC Remote Procedure Call

SHA Secure Hashing Algorithm

UI User Interface

Chapter 1

Introduction

CHAPTER 1

INTRODUCTION

This chapter frames the problem of secure and anonymous electronic voting and explains why a blockchain-backed approach is explored. It situates the project within the context of privacy risks, auditability requirements, and accessibility needs. It also introduces the local-chain approach as a practical implementation path for edge environments. The chapter concludes by defining the scope of the work and how the proposed flow addresses key limitations.

1.1 Introduction

Electronic voting systems have evolved significantly over recent decades, yet the twin challenges of preserving voter privacy and ensuring result integrity remain largely unsolved. Many deployed systems continue to rely on centralized databases or direct vote logging, which exposes voters to identity-to-choice linkage and creates single points of failure for tampering. Blockchain technology offers immutability and public auditability as potential remedies, but a naive on-chain design introduces timing correlations and direct vote mapping that undermine anonymity. This project investigates a hybrid architecture that combines identity hashing, batched and shuffled submission, and local-chain deployment to address these gaps while remaining practically deployable.

1.2 Literature Review

A literature review surveys the current body of knowledge relevant to a topic, synthesizing substantive findings and theoretical or methodological contributions from prior work. The following reviews examine research in blockchain standardization, authentication protocols, verifiable online voting, and privacy-preserving data collection — areas that collectively inform the design of the proposed system. Each paper is reviewed for its core findings and their relevance to the present project.

1.2.1 Review of Related Works

Blockchain Standardization and Regulation

Fardad and Mohammadzadeh Mianji [1] examine the landscape of blockchain and distributed ledger technology standardization...

Authentication in Constrained Systems

Hosseinzadeh and Servati [2] evaluate two proposed ultra-lightweight RFID authentication protocols...

End-to-End Verifiable Online Voting

Alsadi and Casey [3] address the lack of transparency and verifiability...

Differential Privacy on Blockchain

Wang and Zhu [4] contrast private and public blockchain architectures...

1.3 Motivation

Privacy-preserving voting is essential for democratic integrity and public trust...

1.4 Problem Statement

The core problem is to prevent linkage between voter identity and candidate choice...

1.5 Objectives

The objective of this project is to design and implement a privacy-preserving electronic voting system...

1.6 Brief Methodology of the project

1. Voter identity is hashed at registration to decouple identity from the voting record.
2. Cast votes are stored in a local buffer before submission.
3. Votes are shuffled and submitted as a batch to reduce timing correlation.
4. The batch is written to the local blockchain to ensure immutability and auditability.
5. Aggregated results are displayed without exposing individual vote mappings.

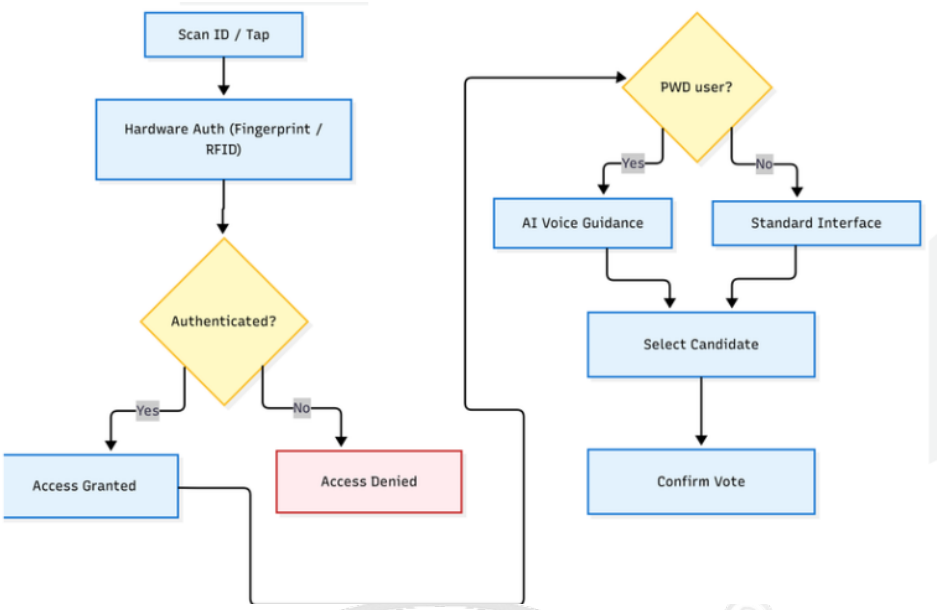

Figure 1.1: Methodology Flow

1.7 Assumptions made / Constraints of the project

The following assumptions and constraints apply to the current phase of the project:

- The system operates on a local (private) blockchain; public chain deployment is outside the current scope.
- Hardware-backed identity verification (e.g., biometrics, smart cards) is not included in this phase.
- The voter device is assumed to be trusted; client-side malware is outside the threat model.
- Network connectivity is assumed only within the local deployment environment.
- The prototype is evaluated for functional correctness and anonymity properties; formal cryptographic proofs are deferred to future work.

1.8 Organization of the Report

This report is organized as follows.

- Chapter 1 introduces the project, reviews relevant literature, states the problem, and defines the scope and objectives.

- Chapter 2 discusses the fundamentals of blockchain technology and the privacy properties relevant to electronic voting.
- Chapter 3 discusses the system architecture, including identity hashing, batch shuffling, and local-chain design.
- Chapter 4 discusses the implementation of the voting flow and the dual-interface client.
- Chapter 5 discusses future work and scope of the project.

Chapter 2

Design of Blockchain-Based Voting Architecture

CHAPTER 2

DESIGN OF BLOCKCHAIN-BASED VOTING ARCHITECTURE

Secure and anonymous electronic voting demands a careful separation between identity verification and vote recording. This chapter presents the architectural components of the proposed system and traces the end-to-end flow through which voter privacy is preserved. The design rationale for off-chain batching, identity hashing, and local blockchain deployment is elaborated across the following sections. Together, these elements form a modular pipeline that achieves anonymity without sacrificing auditability.

2.1 System Overview

The proposed system is composed of four principal layers: a voter-facing User Interface (UI), an off-chain batching and shuffle module, a relayer service, and an on-chain smart contract responsible for tallying. Voters authenticate at the UI, select a candidate, and submit a vote that enters an off-chain buffer rather than being written directly to the chain. This buffering strategy is central to the privacy model — votes accumulate until a threshold batch size is reached, at which point they are shuffled and submitted in two distinct stages. By ensuring that hashed identities are marked used in one transaction and anonymized candidate Identitys (IDs) are recorded in a separate transaction, the system prevents any direct on-chain pairing of a voter to a choice.

2.1.1 Voter Interface

The UI layer provides two interaction modes. The standard interface presents the ballot visually, allowing voters to select a candidate and confirm their submission through conventional controls. The voice-guided agentic interface targets voters who require assistance, offering a step-by-step spoken flow with explicit confirmation gates at each stage to prevent accidental submission. Both interfaces share the same underlying authentication and submission pipeline, ensuring consistent privacy properties regardless of the mode used.

2.1.2 Off-Chain Buffer and Shuffle Module

The off-chain buffer temporarily holds incoming votes as (hashed identity, candidate ID) pairs. Once the buffer accumulates a configurable threshold of entries, the module

applies a shuffle to the list of candidate IDs, decoupling their ordering from the order in which votes were cast. This shuffle is the primary mechanism for breaking timing correlations that could otherwise allow an observer to infer which voter chose which candidate by comparing submission timestamps.

2.1.3 Relayer Service

The relayer is the sole actor authorized to submit batched transactions to the smart contract. It receives the shuffled batch from the off-chain module and performs a two-stage submission: first it forwards the array of hashed identities to mark them as used, and then it forwards the array of anonymized candidate IDs to update the tally. This separation into two transactions is a deliberate design choice — even if both transactions are visible on-chain, the shuffle ensures no positional correspondence exists between the two arrays.

2.1.4 Smart Contract

The smart contract enforces election lifecycle phases (registration, voting, closed), validates submitted candidate IDs against a declared candidate list, rejects any hashed identity that has already been marked as used, and maintains running candidate tallies. All state transitions are recorded immutably on the local chain. The contract exposes a read-only results function that returns aggregate counts without any per-voter data, supporting public auditability while preserving individual anonymity.

2.2 System Flow Description

Authentication and Identity Hashing

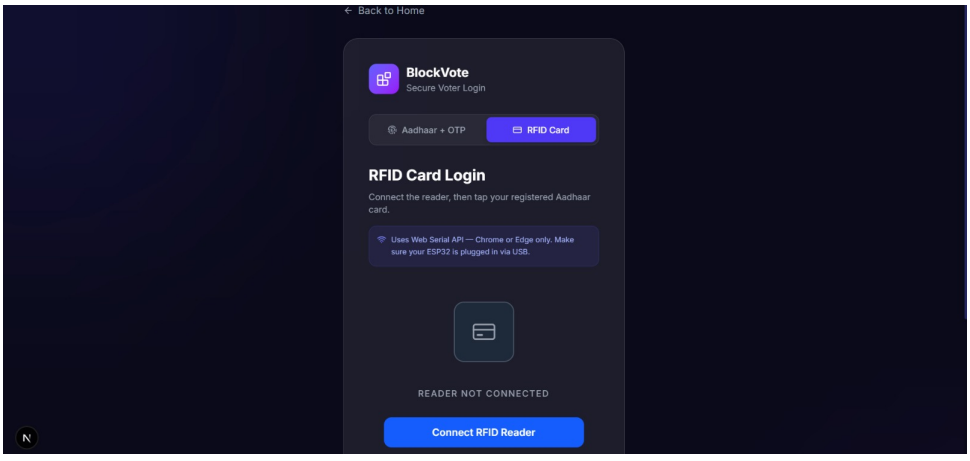

Figure 2.1: Authentication and Identity Hashing

When a voter initiates a session, the UI collects the identity credential and immediately computes a deterministic hash. The raw credential is discarded after hashing and is never stored, transmitted, or written to any persistent medium. The hash is used solely to verify eligibility and, once the vote is submitted, to mark the voter as having participated.

Candidate Selection

Following authentication, the voter selects a candidate. In the standard interface, selection is made via a visual ballot control. In the voice-guided flow, the system reads out candidate names and awaits a spoken or keyed confirmation at each decision point. A final confirmation gate is presented in both modes before the vote is committed to the buffer.

Buffering and Shuffle

The confirmed (hashed identity, candidate ID) pair is appended to the off-chain buffer. The buffer continues to accept incoming votes until its threshold is reached. At threshold, the candidate ID array is shuffled independently of the identity array. This shuffle is applied only to the candidate column; the identity column retains its original order for the first-stage submission. After the shuffle, the two arrays no longer share a meaningful positional mapping.

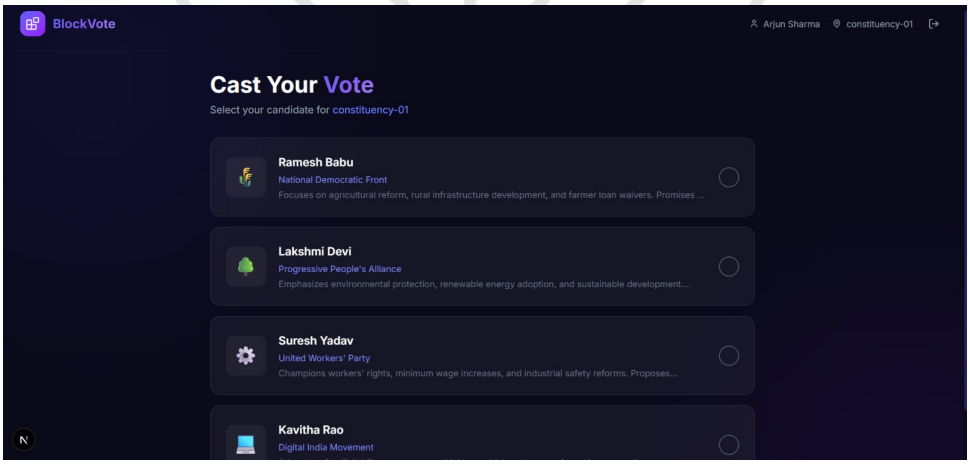

Figure 2.2: Candidate Selection

Two-Stage On-Chain Submission

The relayer submits the identity array to the smart contract in the first transaction, which marks each hash as used and enforces the no-double-vote constraint. The relayer

then submits the shuffled candidate ID array in a second transaction, which increments the corresponding tally counters. Because the two transactions carry independently ordered arrays, no on-chain observer can reconstruct which identity corresponded to which candidate selection.

2.3 Data Handling and Privacy

Raw identity data is never retained beyond the hashing step. The off-chain buffer holds only hashed identities and candidate IDs, and both are discarded once the batch has been successfully submitted to the chain. On-chain state consists exclusively of a set of used identity hashes and a mapping of candidate IDs to vote counts. No per-vote record, timestamp, or sequence number that could enable re-identification is written to the ledger.

2.3.1 Threat Model Considerations

The design addresses two primary threat classes. The first is linkage attack: an adversary attempts to pair a voter identity with a candidate choice by correlating on-chain data. The shuffle step and the two-stage submission together eliminate the positional mapping required for this attack. The second is double-vote fraud: a voter or a compromised relayer attempts to register more than one vote for the same identity. The smart contract's duplicate-rejection logic defeats this by treating the identity hash set as a spent-key register.

2.3.2 Local Blockchain Rationale

Deployment on a local (private) chain rather than a public network provides two practical benefits. First, it removes the dependency on external validators and network fees, making the system viable for edge environments with limited or intermittent connectivity. Second, it constrains the visible surface of on-chain data to participants within the deployment boundary, reducing exposure to passive chain analysis. The modular architecture is designed so that migration to a public or permissioned chain in a future phase requires changes only to the deployment configuration, not to the application logic.

2.4 Tools and Technologies Used

2.4.1 Smart Contract Platform

The smart contract is implemented in Solidity and deployed to a local Electronic Voting Machine (EVM)-compatible chain. A local node instance provides the Remote Procedure Call (RPC) endpoint consumed by the relayer. This setup allows the full contract lifecycle — deployment, interaction, and state inspection — to be exercised without external dependencies.

2.4.2 Off-Chain Components

The off-chain buffer, shuffle module, and relayer are implemented as a lightweight service with a REpresentational State Transfer (REST) interface. The service maintains buffer state in memory and exposes endpoints for vote submission (called by the UI) and batch flush (triggered internally at threshold). The shuffle algorithm uses a cryptographically seeded Fisher-Yates permutation to ensure unpredictability of the resulting order.

2.4.3 User Interface Framework

The standard UI is built as a web-based single-page application. The voice-guided interface uses a browser-native speech Application Programming Interface (API) for output and a text input fallback for environments where speech recognition is unavailable. Both interfaces communicate with the off-chain service over HyperText Transfer Protocol Secure (HTTPS).

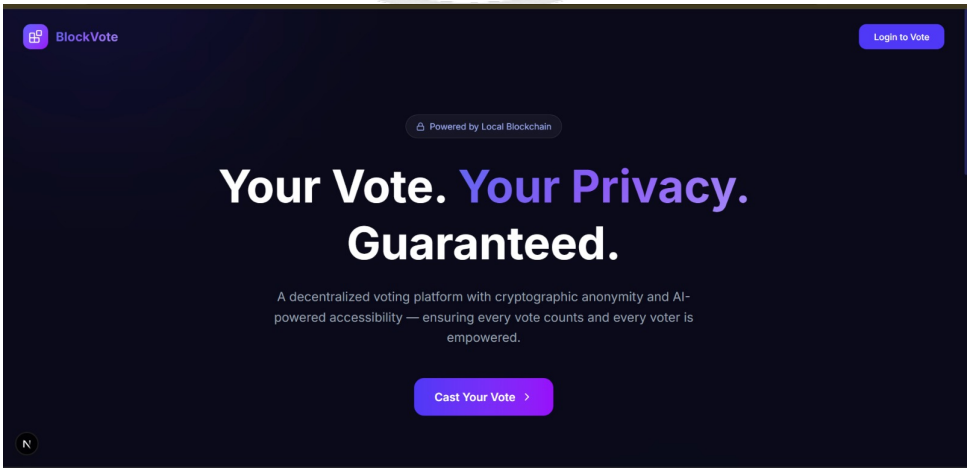

Figure 2.3: Standard Visual Interface

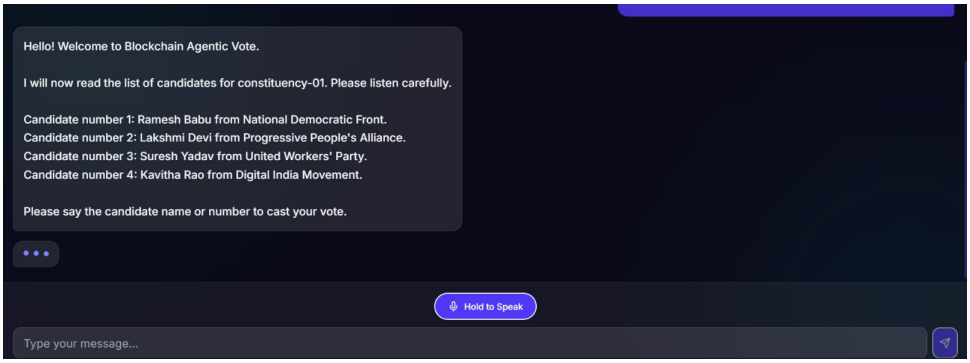

Figure 2.4: Voice-Guided Interface

2.5 Use of Acronyms and Glossaries

Acronyms used throughout this report are defined at first use and thereafter appear in their short form. For example, UI, EVM, RPC, REST, and API are defined in the `Glossaries.tex` file and referenced using `\gls{<Ref>}`. The first occurrence of each acronym expands to its full form with the abbreviation in parentheses; subsequent occurrences show only the abbreviation. All acronym definitions follow the syntax:

```
%\newacronym{<Ref>}{<Short-Form>}{<Expanded word>}
\newacronym{ui}{UI}{User Interface}
\newacronym{evm}{EVM}{Ethereum Virtual Machine}
\newacronym{rpc}{RPC}{Remote Procedure Call}
\newacronym{rest}{REST}{Representational State Transfer}
\newacronym{api}{API}{Application Programming Interface}
```

This chapter has presented the full system architecture of the proposed blockchain-based voting system, covering the UI layer, off-chain batching and shuffle pipeline, relayer service, and on-chain smart contract. The two-stage submission model and the local-chain deployment strategy together address the core requirements of voter anonymity and result auditability without reliance on centralized infrastructure. The following chapter details the implementation of each component, describing the specific code structure, contract functions, and the interface flows for both standard and voice-guided voting modes.

Chapter 3

System Methodology and Implementation

CHAPTER 3

SYSTEM METHODOLOGY AND IMPLEMENTATION

A privacy-preserving voting system requires deliberate choices at every layer of the stack, from smart contract logic to the off-chain processing pipeline. This chapter details the technology stack, smart contract design, off-chain batching workflow, and the rationale for local blockchain deployment. The design decisions at each stage are motivated by the dual requirements of voter anonymity and result integrity.

3.1 Specifications for the Design

The system is specified as a software-only prototype implementing the complete voting workflow within a local blockchain environment. Key functional requirements include: voter authentication through one-way identity hashing with no raw credential retained, off-chain vote buffering and shuffling prior to on-chain submission, smart contract enforcement of no-double-vote constraints, and a read-only results interface returning only aggregate tallies. Non-functionally, the system must deploy on a local EVM-compatible chain without external dependencies and remain modular enough to migrate to a public chain through configuration changes alone.

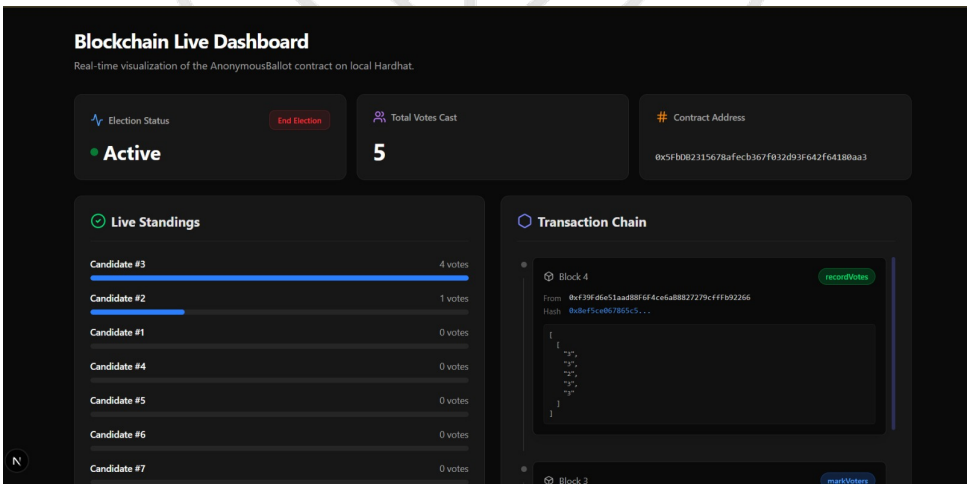

Figure 3.1: Blockchain live dashboard

3.2 Technology Stack and Pre-Analysis

3.2.1 Threat Model

Two primary threat classes were identified prior to design. A *linkage attack* attempts to correlate on-chain data to reconstruct voter-to-candidate mappings; this is mitigated by separating identity and candidate submissions into two independently shuffled arrays [3]. A *double-vote attack* attempts to reuse an identity hash; this is defeated by the smart contract's spent-key register, which rejects any batch containing a previously used hash.

3.2.2 Technology Stack

A JavaScript single-page application provides the voter-facing UI in two modes: a standard visual ballot and a voice-guided agentic flow using the Web Speech API. A lightweight REST backend manages the vote buffer, shuffle module, and relayer. The smart contract is written in Solidity and deployed to a local EVM-compatible development chain, with the relayer as the sole actor submitting batched transactions via RPC.

3.3 Design Methodology

3.3.1 Smart Contract Logic

The contract follows a three-phase state machine: `Registration`, `Voting`, and `Closed`. It exposes two write functions called sequentially by the relayer: `markIdentitiesUsed(bytes32[] identities)`, which checks each hash against the spent-key register and reverts on any duplicate, and `recordVotes(uint8[] candidateIds)`, which increments tally counters. A public `getResults()` function returns aggregate counts without any per-voter data.

3.3.2 Off-Chain Processing and Design Equations

Votes are buffered as (hashed identity, candidate ID) pairs. When the buffer reaches threshold θ, the candidate ID array is shuffled using a Fisher-Yates permutation before submission. The identity hash is computed as:

$$h_i = H(v_i), \quad H : \{0,1\}^* \to \{0,1\}^{256} \tag{3.1}$$

where H is Secure Hashing Algorithm (SHA)-256. The flush condition is $n \geq \theta$, and the shuffle draws a permutation π uniformly from $\mathcal{S}_\theta$, ensuring no positional correspondence between the identity and candidate arrays on-chain [4].

Figure 3.2: Off-chain batching and shuffle workflow prior to on-chain submission

Figure 3.2 illustrates the five-step off-chain workflow: vote submission to the buffer, threshold check, Fisher-Yates shuffle of the candidate column, relayer submission of the identity array, and relayer submission of the shuffled candidate array. The buffer is cleared after each successful batch, and the on-chain record carries no structural information that could reconstruct vote-to-voter mappings.

3.4 Local Chain Rationale

Deployment on a local chain rather than a public network eliminates dependencies on external validators, removes gas fee uncertainty, and restricts on-chain data visibility to participants within the deployment boundary. These properties make the system viable for edge environments with limited connectivity. The modular architecture ensures that migrating to a public or permissioned chain in a future phase requires only reconfiguration of the relayer's RPC target and the contract deployment address.

This chapter has presented the methodology underlying the proposed system, covering the technology stack, contract design, off-chain shuffle workflow, and local chain deployment rationale. The Fisher-Yates shuffle combined with two-stage batch submission implements the unlinkability property central to the privacy design. The following chapter presents the implementation details and test results, evaluating tally correctness, double-vote prevention, and the effectiveness of the shuffle in breaking observable ordering patterns.

Chapter 4

Prototype Implementation and Validation

CHAPTER 4

PROTOTYPE IMPLEMENTATION AND VALIDATION

The software prototype brings together all architectural components described in the preceding chapters into a testable, end-to-end voting system. Completed modules span the voter UI, agentic voice guidance, off-chain batching and shuffle logic, and the on-chain smart contract. This chapter describes each implemented module, the security and anonymity properties demonstrated, and the validation cases used to confirm correct behaviour. The current scope is confined to software on a local blockchain; hardware integration is identified as a future work item.

4.1 Implemented Modules

4.1.1 Voter Interface and Agentic Flow

The standard voting UI presents a candidate list, captures selection, and routes the submission through the off-chain batching endpoint. The agentic voice-guided flow reads candidates aloud using the Web Speech API, prompts for confirmation at each step, and provides neutral responses to reduce selection errors and improve accessibility for voters requiring assistance. Both flows converge at the same batching pipeline, ensuring identical privacy properties regardless of interaction mode. Admin and results views read on-chain state directly, providing transparent access to election phase and aggregate tallies.

4.1.2 Off-Chain Batching and Shuffle Module

The off-chain service accepts (hashed identity, candidate ID) pairs from the UI and appends them to an in-memory buffer. When the buffer reaches the configured threshold θ, the service applies a Fisher-Yates shuffle to the candidate ID column and triggers the relayer. The relayer submits the identity array to `markIdentitiesUsed` and the shuffled candidate array to `recordVotes` in two separate transactions. The buffer is cleared after successful submission.

4.1.3 Smart Contract

The deployed Solidity contract enforces the full election lifecycle. Phase transitions between `Registration`, `Voting`, and `Closed` are owner-controlled and guard all write

functions. Candidate validation rejects any ID outside the declared list. The spent-key register prevents double voting by reverting any batch that contains a previously used identity hash. Tallies are stored as a candidate-to-count mapping and exposed through a public read function.

4.2 Security and Anonymity Coverage

Figure 4.1: Security and anonymity coverage across system layers

Figure 4.1 maps each security property to the layer that enforces it. Identity is hashed at the point of capture and the raw credential is discarded immediately; it is never written to any persistent store. Unlinkability is achieved through the combination of batch accumulation, Fisher-Yates shuffle, and two-stage on-chain submission, ensuring no positional correspondence between identity and candidate arrays exists on-chain [3]. Double-vote prevention is enforced deterministically by the contract's spent-key register. On-chain tallies and explicit phase states provide integrity and auditability. The relayer remains a trust boundary in the current phase: it is the sole actor with write access to the contract, and its correct behaviour is assumed rather than cryptographically enforced. Eliminating this assumption through threshold signing or a verifiable relay protocol is identified as future work.

4.3 Validation Status

Validation was conducted in a local blockchain environment using a development chain node. Test cases covered the following scenarios: repeated voting attempts using the same identity hash, submission of batches containing invalid candidate IDs, batch submission at and below the threshold, and election phase transition enforcement. In all cases the contract reverted disallowed operations and accepted valid batches correctly. Post-submission inspection of on-chain state confirmed that identity hashes and candidate counts were recorded without any positional or temporal link between them. Tallies were verified to be consistent with the submitted batch contents across multiple election runs. No hardware integration has been performed in this phase; the system operates entirely in software on a local chain.

This chapter has demonstrated that the implemented prototype satisfies the core functional and security requirements defined in Chapter 3. All planned software modules

are complete and validated, with identity unlinkability, double-vote prevention, and tally integrity confirmed through targeted test cases. The following chapter presents the conclusions of the project, discusses the limitations of the current prototype, and outlines directions for future work including hardware-backed identity verification and migration to a permissioned or public blockchain.

Chapter 5

Future Work

CHAPTER 5

FUTURE WORK

The prototype demonstrates end-to-end voting with identity unlinkability, double-vote prevention, and auditable tallies validated on a local blockchain. This chapter presents the validation results against the stated objectives, interprets the findings in terms of privacy and integrity guarantees, and identifies the steps required to evolve the system toward production readiness. The discussion is organised around three future directions: privacy hardening, deployment readiness, and hardware integration.

5.1 Validation Results

5.1.1 Test Cases and Outcomes

Table 5.1 summarises the test cases executed against the software prototype and their outcomes.

Table 5.1: Validation test cases and outcomes

Test Case	**Expected Behaviour**	**Outcome**
Valid batch submission	Tallies incremented correctly	Pass
Repeated identity hash in batch	Contract reverts transaction	Pass
Invalid candidate ID in batch	Contract reverts transaction	Pass
Vote submission below threshold	Buffer holds, no chain write	Pass
Phase transition enforcement	Writes rejected outside Voting phase	Pass
Results query post-submission	Correct aggregate counts returned	Pass

All six test scenarios produced the expected contract behaviour. Tally consistency was confirmed by comparing on-chain counts with the known contents of each submitted batch across multiple election runs.

5.1.2 Anonymity Verification

Post-submission inspection of on-chain state confirmed that no positional or temporal correspondence exists between the identity hash array and the candidate ID array. The on-chain record consists solely of a set of used hashes and a candidate-to-count mapping, with no per-vote entry, sequence number, or timestamp that could support re-identification. This satisfies the unlinkability requirement stated in Equation 3.1 of

Chapter 3.

5.2 Interpretation of Results

The results confirm that the two-stage batch submission model, combined with Fisher-Yates shuffling, effectively removes the on-chain basis for linkage attacks. The spent-key register provides deterministic double-vote prevention without storing any voter-identifiable data beyond the hash. The separation of concerns between the off-chain privacy layer and the on-chain integrity layer is validated as an effective design strategy for this threat model. The relayer remains the principal trust assumption: its correct and non-colluding behaviour is asserted rather than cryptographically proven in the current phase, which is the primary limitation of the prototype.

5.3 Future Work

5.3.1 Privacy Hardening

The most significant open item is eliminating the relayer as a single trust point. Introducing a zero-knowledge proof of shuffle correctness would allow any observer to verify that the candidate array was permuted honestly without the relayer revealing the permutation [4]. Formal verification of the smart contract logic and an independent security audit are also planned to strengthen assurance of the double-vote and phase-guard mechanisms. Additional unlinkability mechanisms, such as commit-reveal schemes or threshold submission, will be evaluated under higher adversarial threat models.

5.3.2 Deployment Readiness

Hardening for real deployments requires strengthened key management for both the relayer signing key and the admin election-control key. Monitoring and structured logging should be added to support operational oversight without exposing sensitive data. Load testing under realistic voter arrival rates is needed to characterise buffer latency and determine appropriate threshold values. Auditability features for independent observers, such as Merkle-proof inclusion of batch hashes, are also planned.

5.3.3 Hardware Integration

The planned hardware path moves identity capture and hashing from the browser to an Radio Frequency IDentifier (RFID)-enabled ESP32-based polling terminal. Hardware-backed keys on the device would protect both voter credential capture and relayer signing, reducing the software attack surface. The terminal architecture is designed to preserve the same privacy guarantees validated in the software prototype: identity is hashed on-device and the raw credential never leaves the hardware boundary. Validating this equivalence through end-to-end testing on the hardware platform is the primary goal of the next project phase.

The prototype has demonstrated that a software-only, local-chain voting system can satisfy core requirements of voter anonymity, double-vote prevention, and auditable tallying. The validation results confirm the effectiveness of the batch shuffle and two-stage submission model against linkage attacks. Future work is prioritised around closing the relayer trust gap through cryptographic proofs, hardening the system for operational deployment, and integrating hardware-backed identity verification to extend the privacy boundary to the physical polling edge.

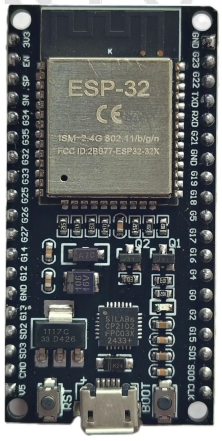

Figure 5.1: Microcontroller ESP32 (a)

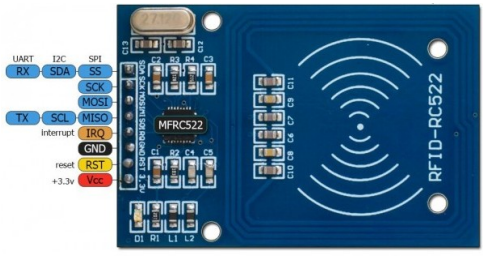

Figure 5.2: Microcontroller ESP32 (b)

Appendix A

Code

APPENDIX A

CODE

A.1 Smart Contract and Batch Processing

A.1.1 AnonymousBallot Contract

The contract stores voter participation and vote tallies in separate mappings so that voter identity and candidate choice are never linked on-chain.

```
SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract AnonymousBallot {
    enum ElectionState { NotStarted, Active, Ended }

    address public immutable admin;
    address public immutable relayer;
    ElectionState public electionState;

    mapping(bytes32 => bool) public hasVoted;
    mapping(uint256 => uint256) public voteCount;
    uint256[] public candidateIds;
    mapping(uint256 => bool) public isValidCandidate;
    uint256 public totalVotes;

    error OnlyAdmin();
    error OnlyRelayer();
    error ElectionNotActive();
    error InvalidStateTransition();
    error VoterAlreadyVoted(bytes32 voterHash);
    error InvalidCandidate(uint256 candidateId);
    error EmptyBatch();

    modifier onlyRelayer() {
```

```
        if (msg.sender != relayer) revert OnlyRelayer();
        _;
    }

    modifier onlyDuringElection() {
        if (electionState != ElectionState.Active) revert
    ElectionNotActive();
        _;
    }

    constructor(address _relayer, uint256[] memory _candidateIds) {
        admin = msg.sender;
        relayer = _relayer;
        electionState = ElectionState.NotStarted;

        for (uint256 i = 0; i < _candidateIds.length; i++) {
            candidateIds.push(_candidateIds[i]);
            isValidCandidate[_candidateIds[i]] = true;
        }
    }

    function advanceElectionState() external {
        if (msg.sender != admin) revert OnlyAdmin();
        if (electionState == ElectionState.NotStarted) {
            electionState = ElectionState.Active;
        } else if (electionState == ElectionState.Active) {
            electionState = ElectionState.Ended;
        } else {
            revert InvalidStateTransition();
        }
    }

    function markVoters(bytes32[] calldata voterHashes)
```

```
        external
        onlyRelayer
        onlyDuringElection
    {
        if (voterHashes.length == 0) revert EmptyBatch();

        for (uint256 i = 0; i < voterHashes.length; i++) {
            if (hasVoted[voterHashes[i]]) revert
   VoterAlreadyVoted(voterHashes[i]);
            hasVoted[voterHashes[i]] = true;
        }
    }

    function recordVotes(uint256[] calldata _candidateIds)
        external
        onlyRelayer
        onlyDuringElection
    {
        if (_candidateIds.length == 0) revert EmptyBatch();

        for (uint256 i = 0; i < _candidateIds.length; i++) {
            if (!isValidCandidate[_candidateIds[i]]) {
                revert InvalidCandidate(_candidateIds[i]);
            }
            voteCount[_candidateIds[i]]++;
        }

        totalVotes += _candidateIds.length;
    }
}
```

A.1.2 Batching Logic

Votes are collected off-chain, deduplicated, shuffled, and then submitted on-chain in two separate transactions.

```
import { flushVotes } from "@/lib/redis";
import { getWritableContract, normalizeBytes32 } from "@/lib/blockchain";
import { secureShuffle } from "@/lib/shuffle";

export async function processBatch(constituencyId: string) {
    const votes = await flushVotes(constituencyId);

    if (votes.length === 0) {
        return { success: true, message: "No votes in buffer.", voterCount:
    0 };
    }

    const seenHashes = new Set<string>();
    const uniqueVotes = votes.filter((v) => {
        const normalizedHash = normalizeBytes32(v.voterHash);
        if (seenHashes.has(normalizedHash)) return false;
        seenHashes.add(normalizedHash);
        return true;
    });

    const voterHashes = uniqueVotes.map((v) =>
    normalizeBytes32(v.voterHash));
    const candidateIds = uniqueVotes.map((v) => v.candidateId);
    const shuffledCandidateIds = secureShuffle(candidateIds);

    const contract = getWritableContract();

    const markTx = await contract.markVoters(voterHashes);
    await markTx.wait();

    const voteTx = await contract.recordVotes(shuffledCandidateIds);
    await voteTx.wait();
```

```
    return {
        success: true,
        message: `Batch of ${votes.length} votes processed and recorded
    on-chain.`,
        voterCount: votes.length,
    };
}
```

A.1.3 Secure Shuffle

This helper uses a cryptographically secure Fisher-Yates shuffle.

```
import crypto from "crypto";

export function secureShuffle<T>(array: T[]): T[] {
    const shuffled = [...array];
    for (let i = shuffled.length - 1; i > 0; i--) {
        const randomBytes = crypto.randomBytes(4);
        const randomIndex = randomBytes.readUInt32BE(0) % (i + 1);
        [shuffled[i], shuffled[randomIndex]] = [shuffled[randomIndex],
    shuffled[i]];
    }
    return shuffled;
}
```

A.1.4 Vote Submission Flow

This snippet shows how a vote enters the off-chain anonymity pool, is checked against on-chain and buffer state, and triggers batch processing when the threshold is reached.

```
import { auth } from "@/auth";
import { hasVoterVotedOnChain } from "@/lib/blockchain";
import { pushVote, hasVoterInBuffer } from "@/lib/redis";
import { processBatch } from "@/actions/batch";
import { z } from "zod";
```

```
const submitVoteSchema = z.object({
  candidateId: z.number().int().positive(),
});

export async function submitVote(candidateId: number) {
  const session = await auth();
  if (!session?.user?.voterHash) {
    return { success: false, message: "Not authenticated. Please log in." };
  }

  const parsed = submitVoteSchema.safeParse({ candidateId });
  if (!parsed.success) {
    return { success: false, message: "Invalid candidate ID." };
  }

  const { voterHash, constituency } = session.user;

  const alreadyVoted = await hasVoterVotedOnChain(voterHash);
  if (alreadyVoted) {
    return { success: false, message: "You have already cast your vote." };
  }

  const inBuffer = await hasVoterInBuffer(constituency, voterHash);
  if (inBuffer) {
    return {
      success: false,
      message: "Your vote is already pending in the anonymity pool. Please
   wait for batch processing.",
    };
  }

  const bufferLength = await pushVote(constituency, {
    voterHash,
```

```
    candidateId: parsed.data.candidateId,
    timestamp: Date.now(),
  });

  const threshold = parseInt(process.env.BATCH_THRESHOLD || "5", 10);
  let batchTriggered = false;

  if (bufferLength >= threshold) {
    processBatch(constituency).catch((err) =>
      console.error("Batch processing failed:", err)
    );
    batchTriggered = true;
  }

  return {
    success: true,
    message: batchTriggered
      ? "Vote submitted! Batch is being processed for maximum anonymity."
      : `Vote submitted to anonymity pool. Waiting for more votes
   (${bufferLength}/${threshold}) before batch processing.`,
    batchTriggered,
  };
}
```